

an automatic leader election process provides high availability across the cluster of servers.

Traditionally, databases have been supporting single data models like key value pairs, graphs, relational, hierarchical, text searches, etc; however, the databases coming out today can support more than one model. MongoDB is one such database that has multi-model capabilities. Even though MongoDB provides geo-spatial and text search capabilities, it's not as good or as up to the mark as Solr or Elastic Search, which make better search engines.

## An analysis of our use case

We currently work in the order management space, where the order data is communicated to almost 120+ applications using 270+ integrations through our middleware.

One of the main components we have implemented in-house is our custom logger, which is a service to log the transaction events, enabling message tracking and error tracking for our system. Most of the messaging is asynchronous. The integrations are heterogeneous, whereby we connect to Oracle, Microsoft SQL relational databases, IBM's MQ, Service Bus, Web services and some file based integrations.

Our middleware processes generate a large number of events in the path through which the order travels within the IT system, and these events usually contain order metadata, a few order attributes required for searching; a status indicating the success, errors, warnings, etc; and in some cases, we store the whole payload for debugging, etc.

Our custom logger framework is traditionally used to store these events in plain text log files in each of the server's local file systems, and we have a background Python job to read these log files and shred them into relational database tables. The logging is fast; however, tracking a message across multiple servers and trying to get a real-time view of the order is still not possible. Then there are problems around the scheduler and the background jobs which need to be monitored, etc. In a cluster with both Prod and DR running in active mode with 16 physical servers, we have to run 16 scheduler jobs and then monitor them to ensure that they run all the time. We can increase the speed of data-fetching using multiple threads or schedule in smaller intervals; however, managing them across multiple domains when we scale our clusters is a maintenance headache.

To get a real-time view, we had rewritten our logging framework with a lightweight Web service, which could write directly to RDBMS database tables. This brought down the performance of the system. Initially, when we were writing to a file on local file systems, the processing speed was around 90-100k messages per minute. Now, with the new design of

writing to a database table, the performance was only 4-5k messages per minute. This was a big trade-off in performance levels, which we couldn't afford.

We rewrote the framework with Oracle AQs in between, wherein the Web service writes data into Oracle AQs; there was a scheduler job on the database, which dequeued messages from AQ and inserted data to the tables. This improved the performance to 10k messages per minute. We then hit a dead end with Oracle database and the systems. Now, to get a real-time view of the order without losing much of the performance, we started looking out at the open source ecosystem and we hit upon MongoDB.

It fitted our use case appropriately. Our need was a database that could take in high performance writes where multiple processes were logging events in parallel. Our query rate of this logging data was substantially lower. We quickly modelled the document based on our previous experience, and were able to swiftly roll out the custom logger with a MongoDB backend. The performance improved dramatically to around 70k messages per minute.

This enabled us to have a near real-time view of the order across multiple processes and systems on a need basis, without compromising on the performance. It eliminated the need for multiple scheduler processes across a cluster of servers and also having to manage each of them. Also, irrespective of how many processes or how many servers our host application scaled to, our logger framework hosted on separate infrastructure is able to cater to all the needs in a service-oriented fashion.

Currently, we are learning through experience. Some of the challenges we faced while adopting MongoDB involved managing the data growth and the need to have a purge mechanism for the data. This is something that is not explicitly available, and needs to be planned and managed when we create the shards. The shard management needs to be improved to provide optimal usage of storage. Also, the replicas and the location of the replicas define how good our disaster recovery will be. We have been able to maintain the infra without much hassle and are looking at the opportunity to roll out this logging framework into other areas, like the product master or customer master integration space in our IT. This should be possible without much rework or changes because of MongoDB's flexible JSON document model. 

**By: Raj Thilak and Gautham D.N.**

Raj Thilak is a lead developer at Dell Technologies. He can be reached at [Raj\\_Thilak\\_R@Dell.com](mailto:Raj_Thilak_R@Dell.com).

Gautham D.N. is an IT manager at Dell Technologies. He can be contacted at [Gautham.D.N@dell.com](mailto:Gautham.D.N@dell.com)